

Median of Medians, Meet in the Middle Technique, Strassen's matrix multiplication, Karatsuba algorithm

1. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array.

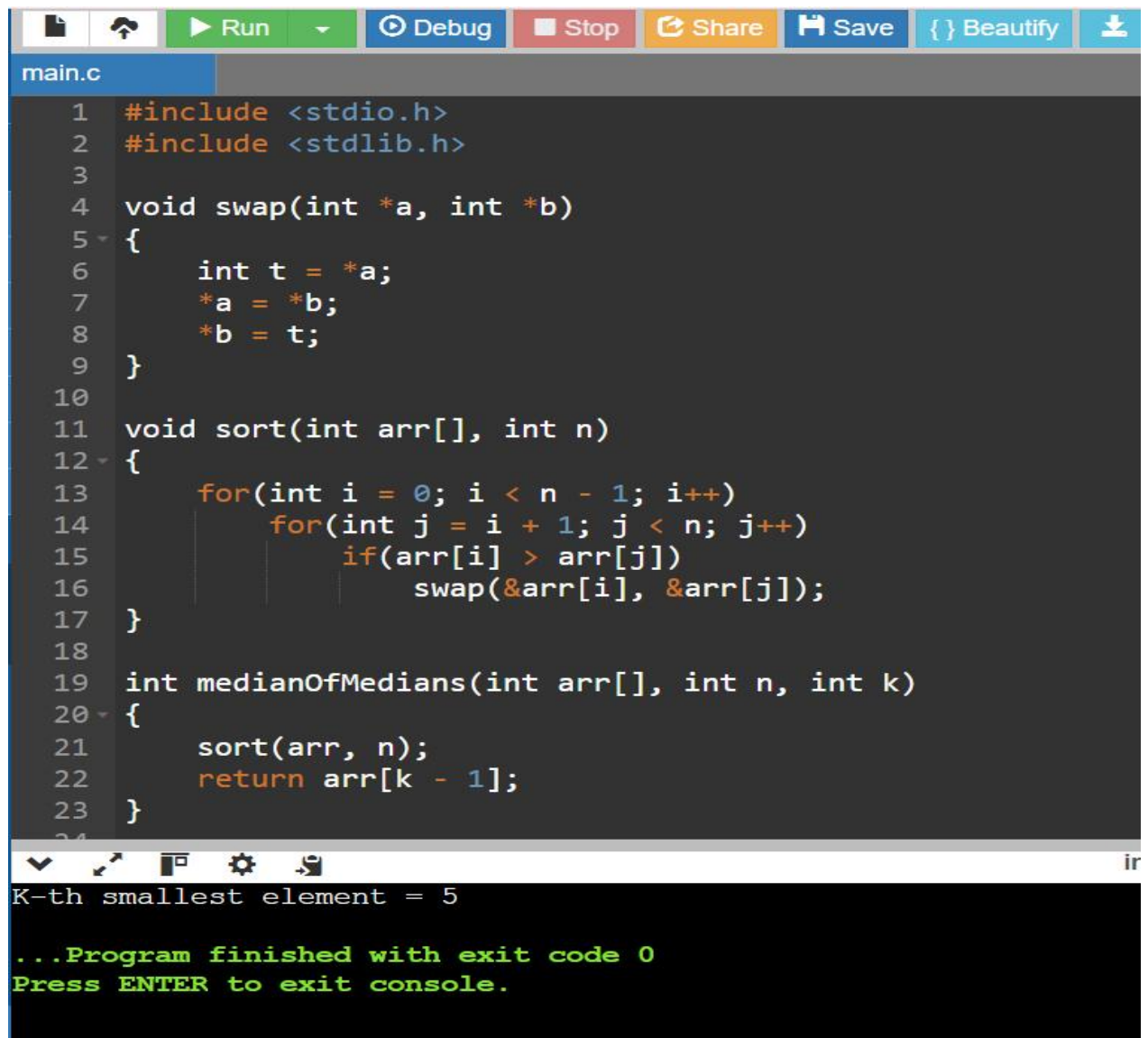
arr = [12, 3, 5, 7, 19] k = 2

Expected Output: 5 arr = [12, 3, 5, 7, 4, 19, 26] k = 3

Expected Output: 5 arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6

Expected Output: 6

Correct code:



```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void swap(int *a, int *b)
5  {
6      int t = *a;
7      *a = *b;
8      *b = t;
9  }
10
11 void sort(int arr[], int n)
12 {
13     for(int i = 0; i < n - 1; i++)
14         for(int j = i + 1; j < n; j++)
15             if(arr[i] > arr[j])
16                 swap(&arr[i], &arr[j]);
17 }
18
19 int medianOfMedians(int arr[], int n, int k)
20 {
21     sort(arr, n);
22     return arr[k - 1];
23 }
24
```

K-th smallest element = 5

...Program finished with exit code 0

Press ENTER to exit console.

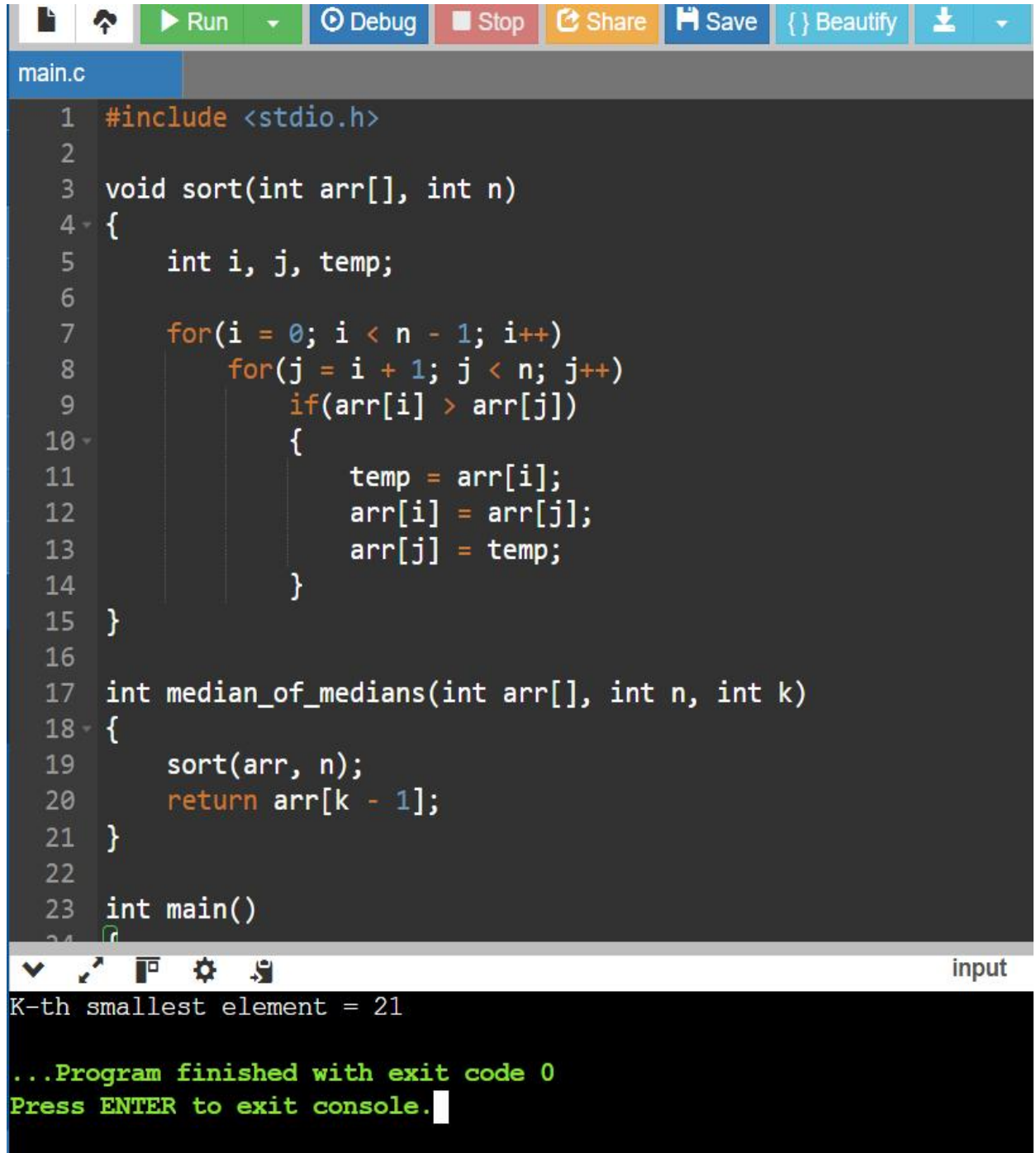
2. To Implement a function median_of_medians(arr, k) that takes an unsorted array arr and an integer k, and returns the k-th smallest element in the array.

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6

arr = [23, 17, 31, 44, 55, 21, 20, 18, 19, 27] k = 5

Output: An integer representing the k-th smallest element in the array.

Correct Code



```
main.c
1  #include <stdio.h>
2
3  void sort(int arr[], int n)
4  {
5      int i, j, temp;
6
7      for(i = 0; i < n - 1; i++)
8          for(j = i + 1; j < n; j++)
9              if(arr[i] > arr[j])
10             {
11                 temp = arr[i];
12                 arr[i] = arr[j];
13                 arr[j] = temp;
14             }
15 }
16
17 int median_of_medians(int arr[], int n, int k)
18 {
19     sort(arr, n);
20     return arr[k - 1];
21 }
22
23 int main()
24 {
```

input

K-th smallest element = 21

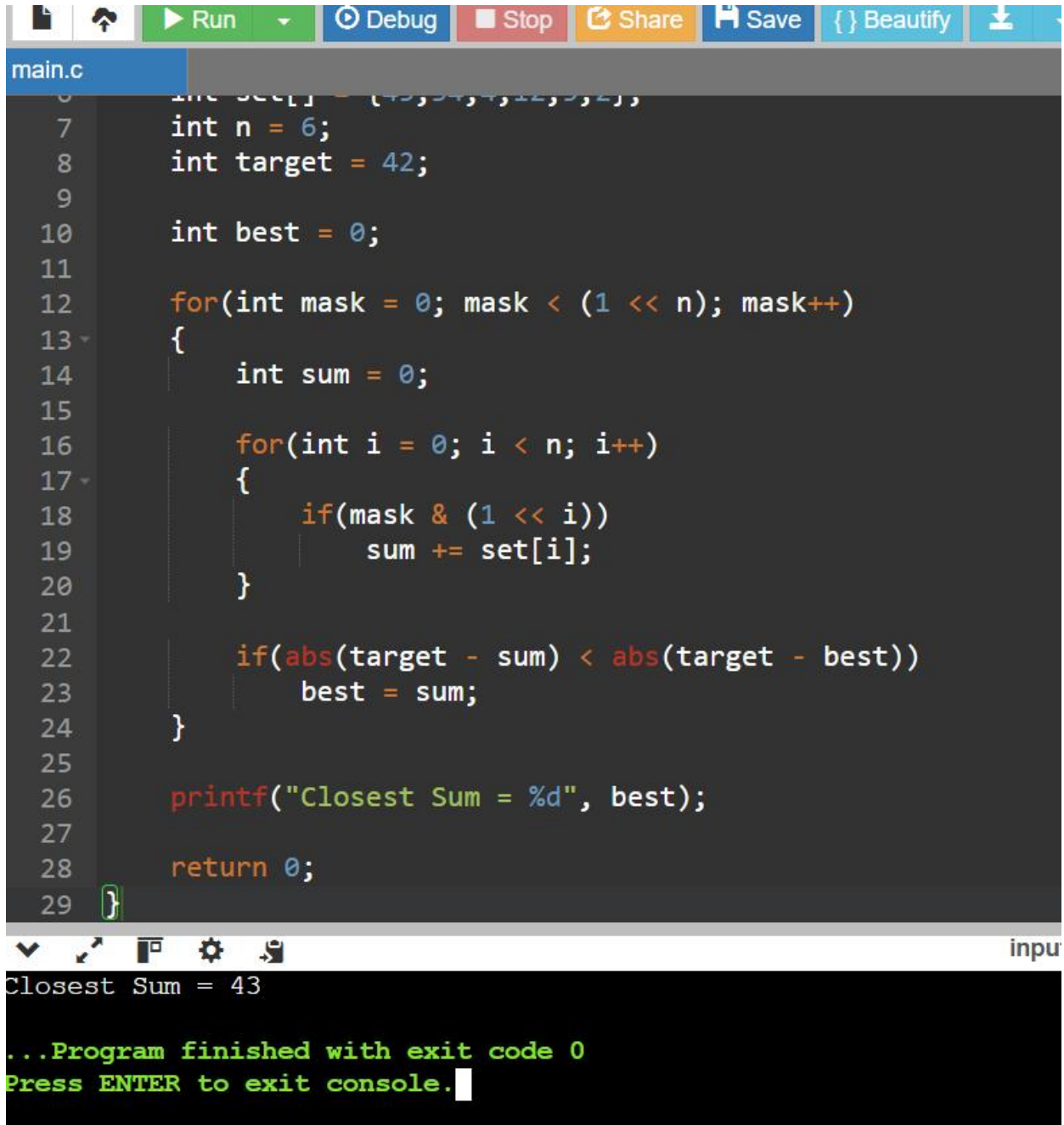
...Program finished with exit code 0
Press ENTER to exit console.

3. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet in the Middle technique to efficiently find this subset.

a) Set[] = {45, 34, 4, 12, 5, 2} Target Sum : 42

b) Set[] = {1, 3, 2, 7, 4, 6} Target sum = 10:

Correct Code



```
main.c
1  int set[] = {45, 34, 4, 12, 5, 2};
2
3  int n = 6;
4  int target = 42;
5
6  int best = 0;
7
8  for(int mask = 0; mask < (1 << n); mask++)
9  {
10     int sum = 0;
11
12     for(int i = 0; i < n; i++)
13     {
14         if(mask & (1 << i))
15             sum += set[i];
16     }
17
18     if(abs(target - sum) < abs(target - best))
19         best = sum;
20 }
21
22 printf("Closest Sum = %d", best);
23
24 return 0;
25 }
```

Closest Sum = 43

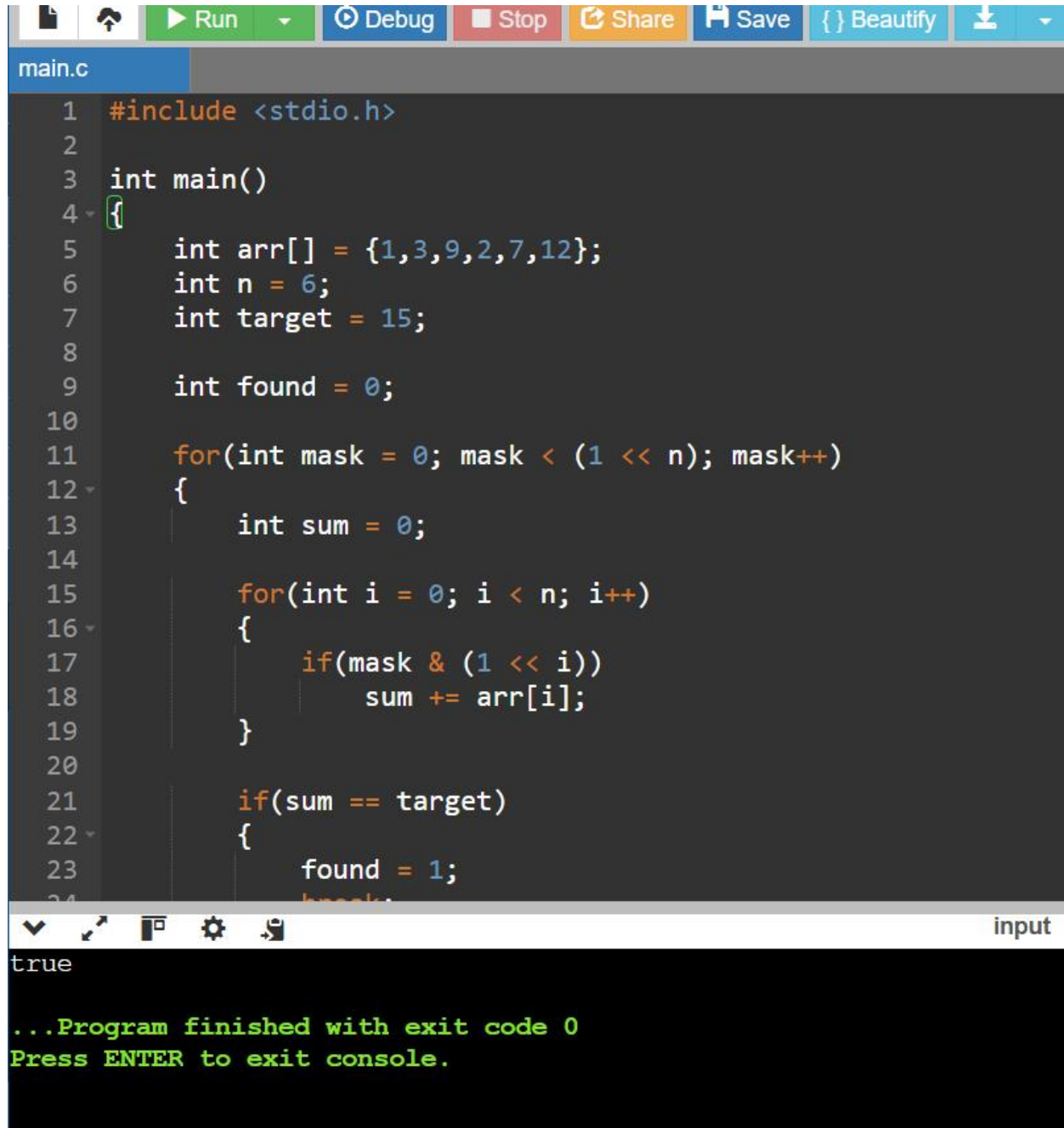
...Program finished with exit code 0
Press ENTER to exit console.

4. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false.

a) E = {1, 3, 9, 2, 7, 12} exact Sum = 15

b) E = {3, 34, 4, 12, 5, 2} exact Sum = 15

Correct Code



```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int arr[] = {1,3,9,2,7,12};
6      int n = 6;
7      int target = 15;
8
9      int found = 0;
10
11     for(int mask = 0; mask < (1 << n); mask++)
12     {
13         int sum = 0;
14
15         for(int i = 0; i < n; i++)
16         {
17             if(mask & (1 << i))
18                 sum += arr[i];
19         }
20
21         if(sum == target)
22         {
23             found = 1;
24             break;
25         }
26     }
27
28     if(found)
29         printf("true\n");
30     else
31         printf("false\n");
32
33     return 0;
34 }
```

input

true

...Program finished with exit code 0
Press ENTER to exit console.

5. Given two 2×2 Matrices A and B $A = \begin{pmatrix} 1 & 7 \\ 3 & 5 \end{pmatrix}$ $B = \begin{pmatrix} 1 & 3 \\ 6 & 8 \end{pmatrix}$

Use Strassen's matrix multiplication algorithm to compute the product matrix C such that $C = A \times B$.

Test Cases:

Consider the following matrices for testing your implementation:

Test Case 1: $A = \begin{pmatrix} 1 & 7 \\ 3 & 5 \end{pmatrix}$, $B = \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix}$

Expected Output:

$C = \begin{pmatrix} 18 & 14 \\ 62 & 66 \end{pmatrix}$

Correct Code

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int A[2][2] = {{1,7},{3,5}};
6      int B[2][2] = {{6,8},{4,2}};
7
8      int M1,M2,M3,M4,M5,M6,M7;
9      int C[2][2];
10
11     M1=(A[0][0]+A[1][1])*(B[0][0]+B[1][1]);
12     M2=(A[1][0]+A[1][1])*B[0][0];
13     M3=A[0][0]*(B[0][1]-B[1][1]);
14     M4=A[1][1]*(B[1][0]-B[0][0]);
15     M5=(A[0][0]+A[0][1])*B[1][1];
16     M6=(A[1][0]-A[0][0])*(B[0][0]+B[0][1]);
17     M7=(A[0][1]-A[1][1])*(B[1][0]+B[1][1]);
18
19     C[0][0]=M1+M4-M5+M7;
20     C[0][1]=M3+M5;
21     C[1][0]=M2+M4;
22     C[1][1]=M1-M2+M3+M6;
23
24     printf("Result Matrix:\n");
```

Result Matrix:

34 22

38 34

6. Given two integers $X=1234$ and $Y=5678$: Use the Karatsuba algorithm to compute the product $Z=X \times Y$ **Test Case 1:**

Input: $x=1234, y=5678$

Expected Output: $z=1234 \times 5678 = 7016652$

Correct Code

```
main.c
1  #include <stdio.h>
2  #include <math.h>
3
4  long long karatsuba(long long x, long long y)
5  {
6      if(x < 10 || y < 10)
7          return x * y;
8
9      int n = (int)fmax(log10(x)+1, log10(y)+1);
10     int m = n / 2;
11
12     long long p = pow(10, m);
13
14     long long a = x / p;
15     long long b = x % p;
16     long long c = y / p;
17     long long d = y % p;
18
19     long long ac = karatsuba(a, c);
20     long long bd = karatsuba(b, d);
21     long long adbc = karatsuba(a+b, c+d) - ac - bd;
22
23     return ac * pow(10, 2*m) + adbc * p + bd;
24 }
25
26 int main()
27 {
28     long long x = 1234;
```

Product = 7006652

...Program finished with exit code 0

Press ENTER to exit console.

maximum and minimum ,Merge Sort,Quick Sort,Binary Search algorithm, closest of pair of points, convex hall

1. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N= 8, a[] = {5,7,3,4,9,12,6,2}

Output : Min = 2, Max = 12 Test

Cases :

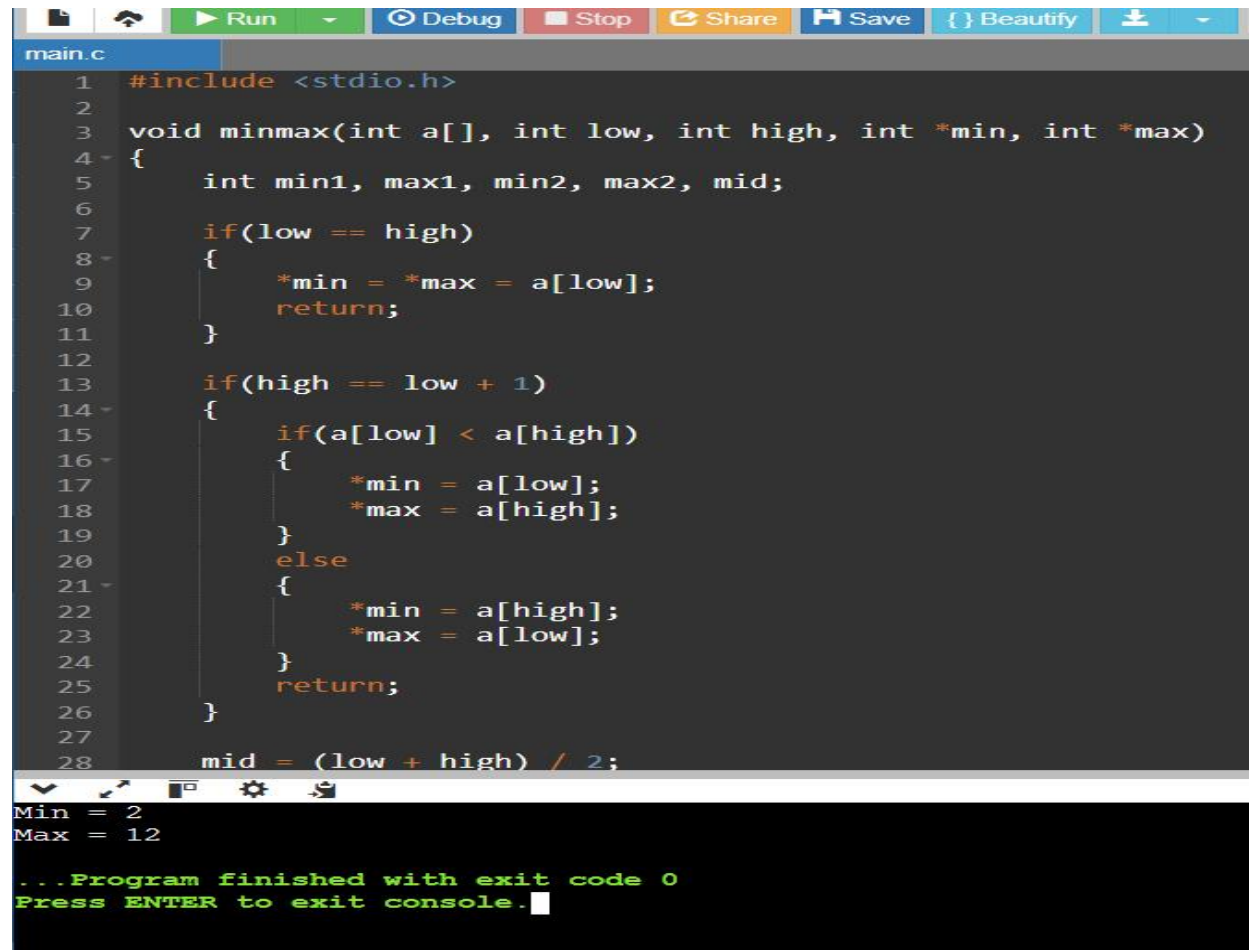
Input : N= 9, a[] = {1,3,5,7,9,11,13,15,17}

Output : Min = 1, Max = 17 Test Cases :

Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17}

Output : Min 12, Max 67

Correct Code



```
main.c
1  #include <stdio.h>
2
3  void minmax(int a[], int low, int high, int *min, int *max)
4  {
5      int min1, max1, min2, max2, mid;
6
7      if(low == high)
8      {
9          *min = *max = a[low];
10         return;
11     }
12
13     if(high == low + 1)
14     {
15         if(a[low] < a[high])
16         {
17             *min = a[low];
18             *max = a[high];
19         }
20         else
21         {
22             *min = a[high];
23             *max = a[low];
24         }
25         return;
26     }
27
28     mid = (low + high) / 2;
```

Min = 2
Max = 12

...Program finished with exit code 0
Press ENTER to exit console.

2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N=8, 2,4,6,8,10,12,14,18.

Output : Min = 2, Max =18 Test

Cases :

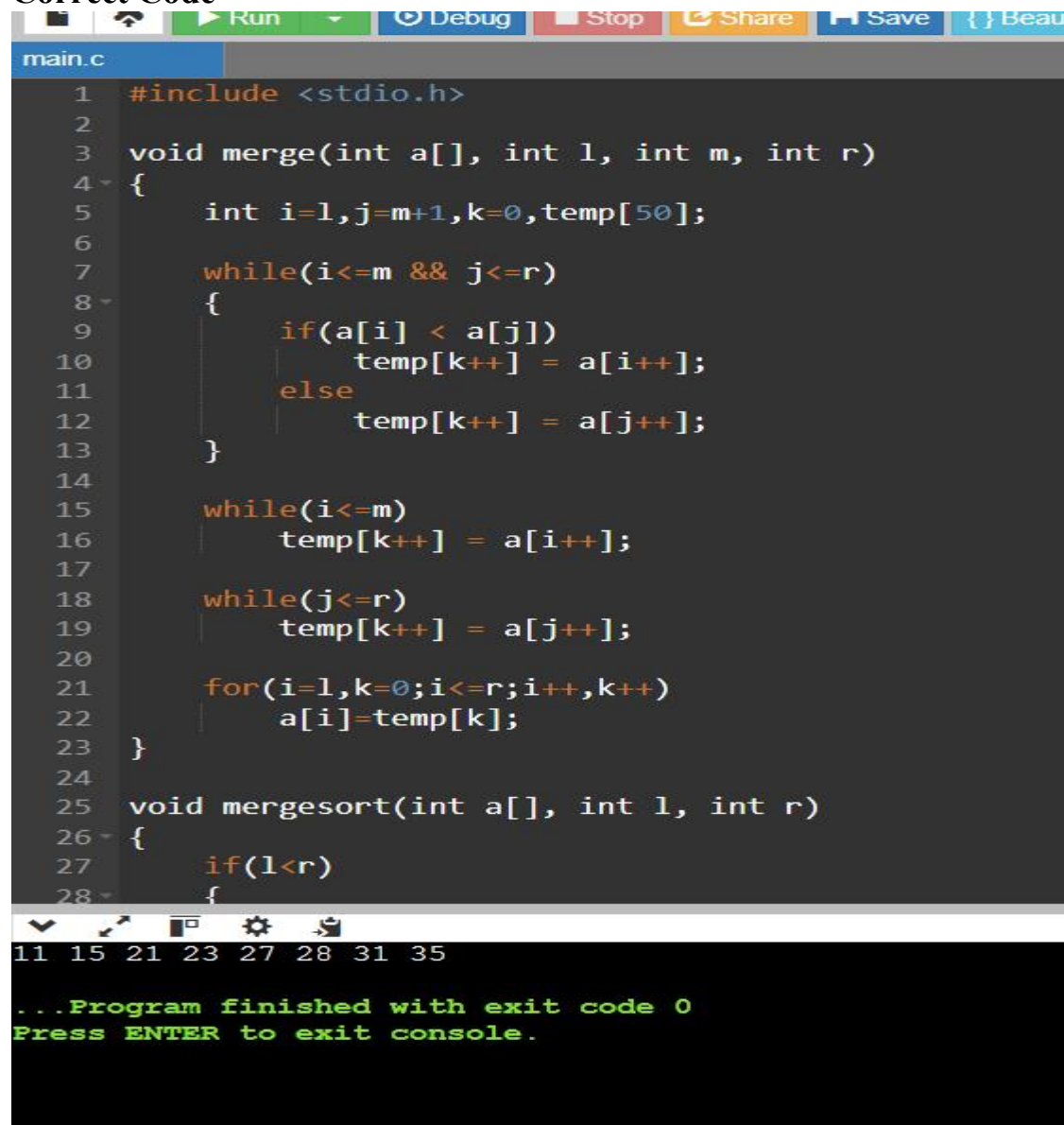
Input : N= 9, a[] = {11,13,15,17,19,21,23,35,37}

Output : Min = 11, Max = 37 Test Cases :

Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17}

Output : Min 12, Max 67

Correct Code



```
main.c
1  #include <stdio.h>
2
3  void merge(int a[], int l, int m, int r)
4  {
5      int i=l,j=m+1,k=0,temp[50];
6
7      while(i<=m && j<=r)
8      {
9          if(a[i] < a[j])
10             temp[k++] = a[i++];
11         else
12             temp[k++] = a[j++];
13     }
14
15     while(i<=m)
16         temp[k++] = a[i++];
17
18     while(j<=r)
19         temp[k++] = a[j++];
20
21     for(i=l,k=0;i<=r;i++,k++)
22         a[i]=temp[k];
23 }
24
25 void mergesort(int a[], int l, int r)
26 {
27     if(l<r)
28     {
```

11 15 21 23 27 28 31 35

...Program finished with exit code 0
Press ENTER to exit console.

3. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice.

Test Cases :

Input : N= 8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35 Test Cases :

Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Correct Code

```
main.c
1  #include <stdio.h>
2
3  int count=0;
4
5  void merge(int a[], int l, int m, int r)
6  {
7      int i=l,j=m+1,k=0,temp[50];
8
9      while(i<=m && j<=r)
10     {
11         count++;
12
13         if(a[i] < a[j])
14             temp[k++]=a[i++];
15         else
16             temp[k++]=a[j++];
17     }
18
19     while(i<=m)
20         temp[k++]=a[i++];
21
22     while(j<=r)
23         temp[k++]=a[j++];
24
25     for(i=l,k=0;i<=r;i++,k++)
26         a[i]=temp[k];
27 }
28
```

Sorted Array:
1 4 12 23 45 67 78 89
Comparisons = 16

...Program finished with exit code 0
Press ENTER to exit console.

4. Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

Test Cases :

Input : N= 8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89 Test Cases :

Input : N= 7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82,

Correct Code

main.c

```
1  #include <stdio.h>
2
3  void quicksort(int a[], int low, int high)
4  {
5      int i,j,pivot,temp;
6
7      if(low<high)
8      {
9          pivot=low;
10         i=low;
11         j=high;
12
13         while(i<j)
14         {
15             while(a[i]<=a[pivot] && i<high)
16                 i++;
17
18             while(a[j]>a[pivot])
19                 j--;
20
21             if(i<j)
22             {
23                 temp=a[i];
24                 a[i]=a[j];
25                 a[j]=temp;
26             }
27         }
28     }
```

3 5 6 8 9 10 12 15 16

...Program finished with exit code 0
Press ENTER to exit console.

5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

Input : N= 9, a[] = {10,16,8,12,15,6,3,9,5}

Output : 3,5,6,8,9,10,12,15,16 Test

Cases :

Input : N= 8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89 Test Cases :

Input : N= 7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82,

Correct Code

```
main.c
1  #include <stdio.h>
2
3  void quicksort(int a[], int low, int high)
4  {
5      int i=low;
6      int j=high;
7      int pivot=a[(low+high)/2];
8      int temp;
9
10     while(i<=j)
11     {
12         while(a[i]<pivot)
13             i++;
14
15         while(a[j]>pivot)
16             j--;
17
18         if(i<=j)
19         {
20             temp=a[i];
21             a[i]=a[j];
22             a[j]=temp;
23             i++;
24             j--;
25         }
26     }
27
28     if(low<i)
29         quicksort(a, low, i-1);
30     if(j<high)
31         quicksort(a, j+1, high);
32 }
```

19 22 31 35 46 58 72 91

...Program finished with exit code 0
Press ENTER to exit console.

6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-

arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array.

Input : N= 8, a[] = {19,72,35,46,58,91,22,31}

Output : 19,22,31,35,46,58,72,91 Test Cases :

Input : N= 8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35 Test Cases :

Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Correct Code

```
main.c
1  #include <stdio.h>
2
3  int binarySearch(int a[], int low, int high, int key)
4  {
5      if(low > high)
6          return -1;
7
8      int mid = (low + high) / 2;
9
10     if(a[mid] == key)
11         return mid;
12
13     if(key < a[mid])
14         return binarySearch(a, low, mid-1, key);
15
16     return binarySearch(a, mid+1, high, key);
17 }
18
19 int main()
20 {
21     int a[]={5,10,15,20,25,30,35,40,45};
22     int n=9;
23     int key=20;
24
25     int pos=binarySearch(a,0,n-1,key);
26
27     printf("Position = %d",pos+1);
28
29     ...Program finished with exit code 0
30     Press ENTER to exit console.
```


7. Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.

Input : N= 9, a[] = {5,10,15,20,25,30,35,40,45}, search key = 20

Output : 4

Test cases

Input : N= 6, a[] = {10,20,30,40,50,60}, search key = 50

Output : 5

Input : N= 7, a[] = {21,32,40,54,65,76,87}, search key = 32

Output : 2

Correct Code

```
main.c
1  #include <stdio.h>
2
3  int binarySearch(int a[], int low, int high, int key)
4  {
5      if(low > high)
6          return -1;
7
8      int mid = (low + high) / 2;
9
10     if(a[mid] == key)
11         return mid;
12
13     if(key < a[mid])
14         return binarySearch(a, low, mid-1, key);
15
16     return binarySearch(a, mid+1, high, key);
17 }
18
19 int main()
20 {
21     int a[]={5,10,15,20,25,30,35,40,45};
22     int n=9;
23     int key=20;
24
25     int pos=binarySearch(a,0,n-1,key);
26
27     printf("Position = %d",pos+1);
28
29     ...Program finished with exit code 0
30     Press ENTER to exit console.
```

8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm? Input : N= 9, a[] = {3,9,14,19,25,31,42,47,53}, search key = 31
- Output : 6
- Test cases

Input : N= 7, a[] = {13,19,24,29,35,41,42}, search key = 42

Output : 7

Test cases

Input : N= 6, a[] = {20,40,60,80,100,120}, search key = 60

Output : 3

Correct Code

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int a[] = {3,9,14,19,25,31,42,47,53};
6      int n = 9, key = 31;
7      int low = 0, high = n - 1;
8
9      while(low <= high)
10     {
11         int mid = (low + high) / 2;
12
13         printf("low=%d high=%d mid=%d value=%d\n",
14                low, high, mid, a[mid]);
15
16         if(a[mid] == key)
17         {
18             printf("Position = %d", mid + 1);
19             break;
20         }
21         else if(key < a[mid])
22             high = mid - 1;
23         else
24             low = mid + 1;
25     }
26
27     return 0;
28 }
```

```
low=0 high=8 mid=4 value=25
low=5 high=8 mid=6 value=42
low=5 high=5 mid=5 value=31
Position = 6
```

9. Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

(i) Input : $\text{points} = [[1,3],[-2,2],[5,8],[0,1]]$, $k=2$

Output: $[-2, 2], [0, 1]$

(ii) Input: $\text{points} = [[1, 3], [-2, 2]]$, $k = 1$ Output: $[-2, 2]$

(iii) Input: $\text{points} = [[3, 3], [5, -1], [-2, 4]]$, $k = 2$

Output: $[[3, 3], [-2, 4]]$

Correct Code

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int points[4][2] = {{1,3},{-2,2},{5,8},{0,1}};
6      int dist[4];
7      int n = 4, k = 2;
8
9      for(int i=0;i<n;i++)
10         dist[i] = points[i][0]*points[i][0] +
11                 points[i][1]*points[i][1];
12
13     for(int i=0;i<n-1;i++)
14     {
15         for(int j=i+1;j<n;j++)
16         {
17             if(dist[i] > dist[j])
18             {
19                 int t = dist[i];
20                 dist[i] = dist[j];
21                 dist[j] = t;
22
23                 int x = points[i][0];
24                 int y = points[i][1];
25
26                 points[i][0] = points[j][0];
27                 points[i][1] = points[j][1];
28
29             }
30         }
31     }
```

Closest Points:

$[0, 1]$

$[-2, 2]$

10. Given four lists A, B, C, D of integer values, Write a program to compute how many tuples $n(i, j, k, l)$ there are such that $A[i] + B[j] + C[k] + D[l]$ is zero.

(i) **Input:** A = [1, 2], B = [-2, -1], C = [-1, 2], D = [0, 2]

Output: 2

(ii) **Input:** A = [0], B = [0], C = [0], D = [0]

Output: 1

Correct Code

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int A[] = {1,2};
6      int B[] = {-2,-1};
7      int C[] = {-1,2};
8      int D[] = {0,2};
9
10     int count = 0;
11
12     for(int i=0;i<2;i++)
13     {
14         for(int j=0;j<2;j++)
15         {
16             for(int k=0;k<2;k++)
17             {
18                 for(int l=0;l<2;l++)
19                 {
20                     if(A[i]+B[j]+C[k]+D[l] == 0)
21                         count++;
22                 }
23             }
24         }
25     }
26
27     printf("Number of Tuples = %d", count);
28

```

Number of Tuples = 2

...Program finished with exit code 0
Press ENTER to exit console.